

Implementation Plan (Phase 2 Roadmap)

If we got the green light tomorrow to build the real robots: what we would buy, in what order we would build it, how we would verify each step, and how long it would take.

1. Hardware sourcing

All prices are rough estimates (USD) and will be confirmed against real supplier quotes before ordering.

Item	Qty	Est. cost	Notes
ESP32 dev board	3	\$15-21	Writer, Executor, outside-network node
Motor driver (L298N / TB6612)	2	\$4-7	One per robot
DC gear motors + wheels + chassis kit	2 sets	\$12-18	Differential drive; a 2WD kit usually bundles all three
MQ-2 gas module	1	\$2-4	Event sensor: gas/smoke anomaly trigger
SW-520D tilt/vibration module	1	\$1-2	Event sensor: collapse cue
HC-SR501 PIR module	1	\$2-3	Event sensor: trapped worker
nRF24L01 radio modules	3	\$4-8	Writer, Executor, outside node (PA+LNA variant if range tests demand it)
Battery (2x 18650 + holder + charger) and regulation	2	\$8-12	5 V rail for motors and MQ-2/PIR; 3.3 V for ESP32 logic
Wires, breadboards, resistors (ADC divider), connectors	-	\$5-8	
Base total		≈ \$55-75	Matches the Phase 1 budget target

Navigation sensing (priced separately, final choice pending):

Both robots need this, because the Executor reuses the Writer's navigation stack (LiDAR + SLAM in the shared entrance frame) to drive to each beacon target.

- **Option A, low cost:** servo-swept distance sensor (ToF or ultrasonic on an SG90 servo), about \$10-15 per robot, plus an IMU (MPU-6050 class) at about \$3-5 per robot. About \$26-40 for both robots. Cheapest, but scans are slow and sparse, so the robot must stop or crawl while scanning.
- **Option B, real SLAM quality:** a low-cost 2D spinning LiDAR (LD06 / YDLIDAR / RPLIDAR A1 class), about \$70-100 per robot, plus the same IMU. About \$146-210 for both robots. This is what our simulation actually models (360 rays, 8 m range), but it would roughly triple the budget.
- **Fallback if neither is affordable:** the simpler ultrasonic (front + right) right-hand wall-follower from our original design.

Compute note: BreezySLAM is a Python library, so SLAM will run on a laptop for both robots. Each ESP32 streams scans and odometry to it and receives motor commands. The Writer and Executor missions run one after the other, so one laptop is enough. Event sensors (MQ-2, tilt, PIR) go on the Writer only; the Executor consumes beacons and does not detect events.

2. Build order

The order is chosen so no step waits on a part we have not tested yet.

- 1 **One robot driving.** Chassis, motors, driver and ESP32 under basic movement control (forward, turn, stop).
- 2 **Perception on that robot.** Wire the LiDAR/IMU, get scans streaming, and run SLAM on the real hardware.
- 3 **Event sensors.** Add MQ-2, tilt switch and PIR, and test each one alone before combining.
- 4 **Radio link.** Add the nRF24L01 and test beacon transmission between two boards.
- 5 **Second robot.** Repeat the same chassis, LiDAR/IMU and SLAM build for the Executor (no event sensors), then add its goal-seeking controller.
- 6 **Outside-network node.** An ESP32 plus radio relay, the simplest build. All robot traffic passes through it (no direct robot-to-command-post link).
- 7 **Full integration test** (see Section 3).

3. Testing plan

Each level has to pass before moving to the next.

- **Bench tests, one component at a time**
 - Gas: baseline reading after preheat, then a clear rise near a gas/smoke source. Verify it returns to baseline.
 - Tilt: triggers on tilt and on a knock; debounce rejects bounce. Check the DO polarity on the actual module.

- PIR: ignores the 30-60 s warm-up, detects a moving person at the set range, and is tested with the robot stopped.
- Motors/IMU: drive a straight line and a 90° turn, and measure the error.
- **Navigation test:** the robot drives a hallway or cardboard maze without wall contact, and returns to its start. Target is a pose error comparable to the simulation (mean 0.14 m, max 0.28 m). We expect real results to be worse and will record how much.
- **Radio test:** beacon sent from one board and received by another, at increasing distance and through obstacles (corners, doors, walls) as a stand-in for a tunnel. We record packet loss against distance. The test also checks the path Writer → outside node → Executor.
- **Shared start frame:** the Writer and Executor both start from the same marked pose at the tunnel entrance, since beacon coordinates only make sense in that shared frame. We repeat the start placement several times and measure how much the placement error shifts the Executor's arrival point.
- **Frame calibration check:** apply the one-time entrance calibration and confirm that a beacon's local position converts to the expected GPS point.
- **Full integration test:** the Writer explores a mock tunnel (cardboard or hallway sections), triggers each event using safe stand-ins (smoke source, tilt, a walking teammate), and drops beacons. The Executor then reaches them using only the beacon trail, stopping within a set arrival radius of each target (0.35 m in simulation). This is the physical version of what our simulation already shows.

4. Rough timeline (about 8 weeks)

Weeks	Focus
1-2	Order parts (allow for shipping delays to Tunisia). Build and drive robot #1.
3-4	LiDAR/IMU on real hardware, SLAM running, event sensors bench-tested.
5	Radio link and beacon transmission; start range tests.
6	Build robot #2 and the outside-network node.
7	Integration testing in the mock tunnel.
8	Fix issues, retest, and record the demo.

This is an estimate, not a fixed schedule. The biggest risks to it are part delivery times, SLAM tuning on real sensor noise, and radio range in real tunnels. We have built a spare week into week 8 for these.